MODUL 6

Queue

Modul Praktikum Struktur Data • C++17

Program Studi Informatika • Praktikum Struktur Data • C++17

1. Tujuan Praktikum

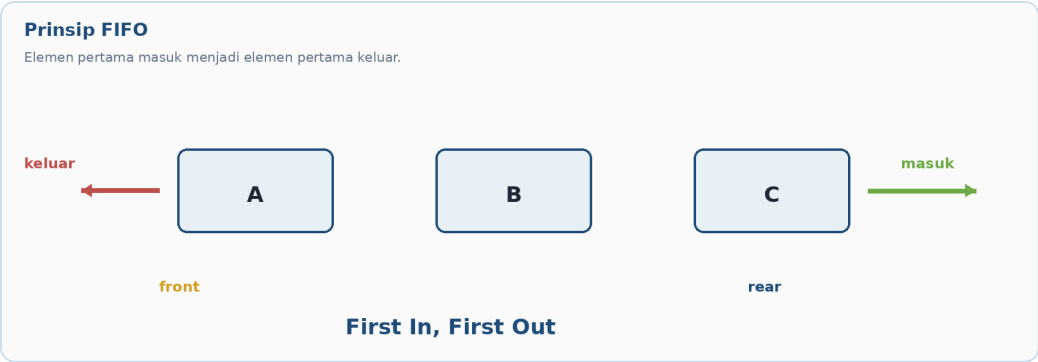
1. menjelaskan prinsip First In, First Out (FIFO);
2. menjelaskan fungsi front, rear, dan count;
3. membedakan queue kosong dan queue penuh;
4. menjelaskan kelemahan linear queue;
5. mengimplementasikan circular queue menggunakan array;
6. melakukan enqueue, dequeue, dan peek;
7. menangani overflow, underflow, dan wrap-around;
8. menampilkan queue tanpa mengubah keadaannya;
9. menggunakan queue pada sistem antrean layanan; dan
10. menjelaskan implementasi alternatif menggunakan linked list.

2. Dasar Queue

Queue menyimpan data dengan aturan FIFO. Data yang pertama dimasukkan menjadi data pertama yang dikeluarkan.

2.1 Prinsip FIFO

Enqueue menambahkan data melalui rear, sedangkan dequeue mengambil data melalui front.



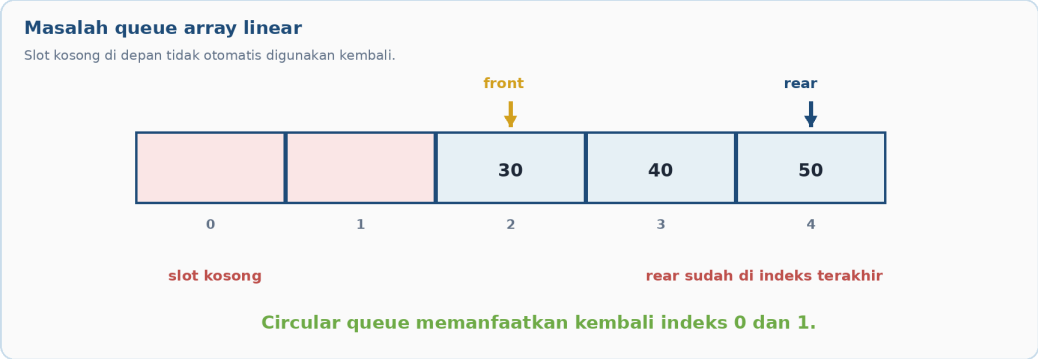
Gambar 1. Prinsip FIFO pada queue.

Operasi	Keterangan
enqueue	Menambahkan data pada bagian belakang

Operasi	Keterangan
dequeue	Mengambil dan menghapus data paling depan
peek	Melihat data paling depan tanpa menghapusnya
isEmpty	Memeriksa apakah queue kosong
isFull	Memeriksa apakah queue array penuh

2.2 Linear queue dan ruang yang tidak terpakai

Pada linear queue, rear terus bergerak menuju indeks terakhir. Ruang kosong di depan tidak dapat digunakan kembali tanpa memindahkan data.

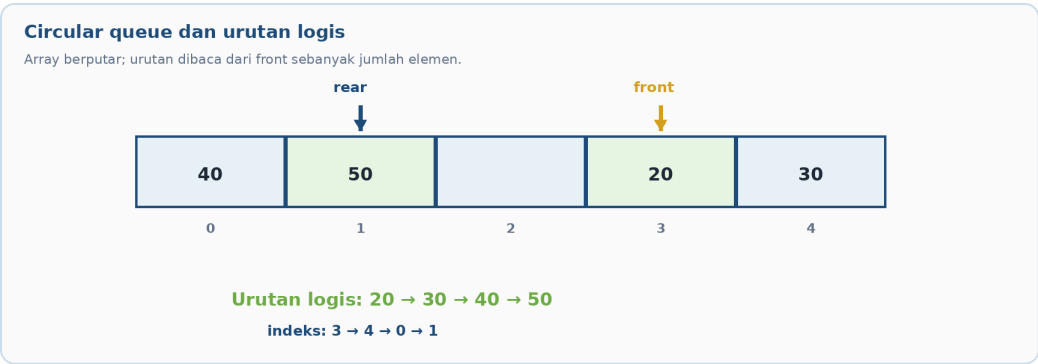


Gambar 2. Ruang di depan linear queue tidak digunakan kembali.

2.3 Circular queue

Circular queue menggunakan kembali indeks awal. Perpindahan front dan rear dilakukan dengan operator modulo.

```
indeksBerikut = (indeks + 1) % KAPASITAS;
```



Gambar 3. Susunan fisik dan urutan logis circular queue.

2.4 Representasi queue

Front menyimpan indeks data pertama, rear indeks data terakhir, dan count jumlah data aktif.

```
const int KAPASITAS = 5;

struct Queue {
    int data[KAPASITAS];
    int front;
    int rear;
    int count;
};
```

2.5 Inisialisasi dan keadaan queue

Queue kosong memiliki count nol. Queue penuh memiliki count sama dengan kapasitas.

```
void init(Queue& q) {
    q.front = 0;
    q.rear = -1;
    q.count = 0;
}

bool isEmpty(const Queue& q) {
    return q.count == 0;
}

bool isFull(const Queue& q) {
    return q.count == KAPASITAS;
}
```

2.6 Kondisi yang harus dijaga

- count berada pada rentang 0 sampai KAPASITAS;
- enqueue menambah count tepat satu;
- dequeue mengurangi count tepat satu;
- front dan rear berpindah secara melingkar;
- peek tidak mengubah front, rear, maupun count;
- enqueue ditolak ketika queue penuh; dan
- dequeue serta peek ditolak ketika queue kosong.

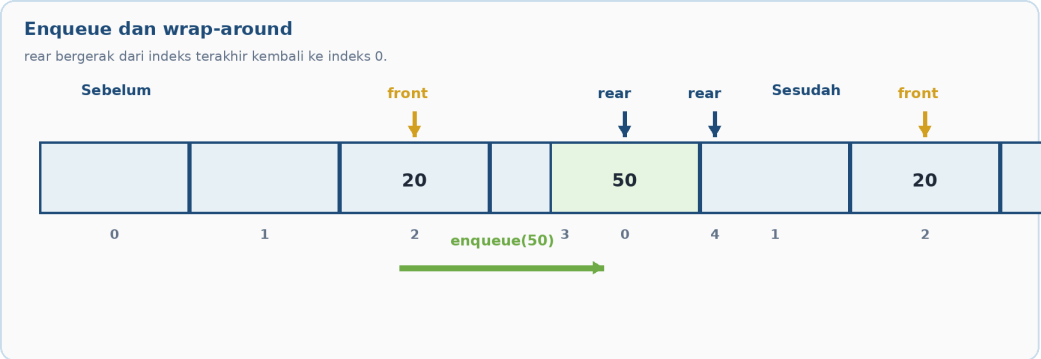
3. Operasi Queue

Operasi queue harus memeriksa keadaan penuh atau kosong sebelum indeks dan count diubah.

3.1 Enqueue

Enqueue memindahkan rear, menyimpan data, kemudian menambah count.

```
bool enqueue(Queue& q, int nilai) {
    if (isFull(q)) return false;
    q.rear = (q.rear + 1) % KAPASITAS;
    q.data[q.rear] = nilai;
    ++q.count;
    return true;
}
```

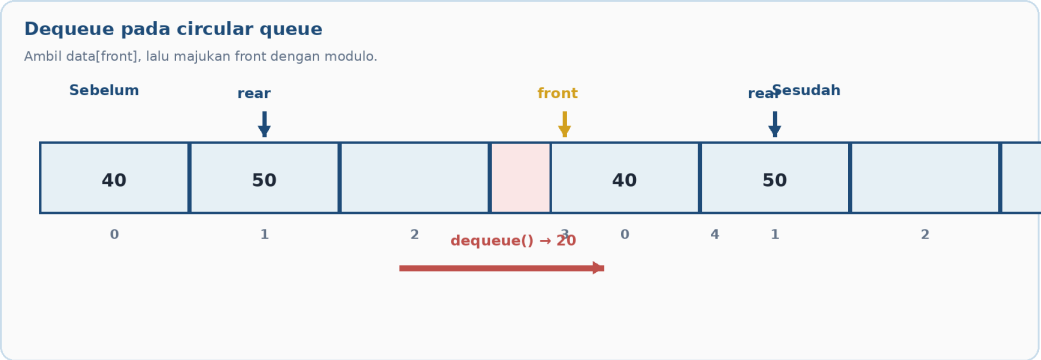


Gambar 4. Rear kembali ke indeks awal saat wrap-around.

3.2 Dequeue

Dequeue mengambil data pada front, memindahkan front, kemudian mengurangi count.

```
bool dequeue(Queue& q, int& nilaiKeluar) {
    if (isEmpty(q)) return false;
    nilaiKeluar = q.data[q.front];
    q.front = (q.front + 1) % KAPASITAS;
    --q.count;
    return true;
}
```



Gambar 5. Dequeue memindahkan front ke data berikutnya.

3.3 Peek

Peek membaca data paling depan tanpa mengubah keadaan queue.

```
bool peek(const Queue& q, int& nilaiDepan) {
    if (isEmpty(q)) return false;
    nilaiDepan = q.data[q.front];
    return true;
}
```

3.4 Menampilkan queue

Urutan logis queue dimulai dari front sebanyak count elemen, bukan dari indeks nol sampai rear.

```
void tampilkan(const Queue& q) {
    if (isEmpty(q)) {
        std::cout << "Queue kosong\n";
        return;
    }
    for (int i = 0; i < q.count; ++i) {
        int indeks = (q.front + i) % KAPASITAS;
        std::cout << q.data[indeks] << ' ';
    }
    std::cout << '\n';
}
```

3.5 Clear, overflow, dan underflow

Clear mengembalikan queue ke keadaan awal. Overflow dan underflow harus ditolak tanpa mengubah front, rear, atau count.

```
void clear(Queue& q) {
    q.front = 0;
    q.rear = -1;
    q.count = 0;
}
```

3.6 Implementasi menggunakan linked list

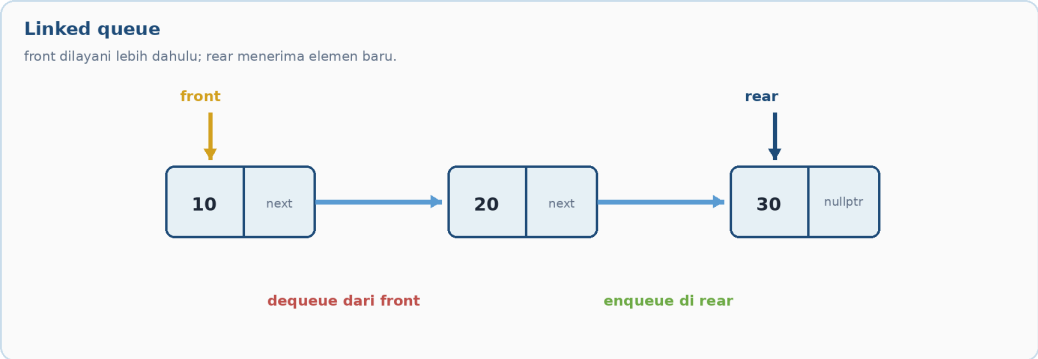
Linked queue menggunakan pointer front dan rear. Ketika node terakhir dikeluarkan, keduanya harus menjadi nullptr.

```
struct Node {
    int data;
    Node* next;
};

struct LinkedQueue {
    Node* front;
    Node* rear;
};

bool enqueue(LinkedQueue& q, int nilai) {
    Node* baru = new Node{nilai, nullptr};
    if (q.rear == nullptr) q.front = q.rear = baru;
    else {
        q.rear->next = baru;
        q.rear = baru;
    }
    return true;
}

bool dequeue(LinkedQueue& q, int& nilaiKeluar) {
    if (q.front == nullptr) return false;
    Node* hapus = q.front;
    nilaiKeluar = hapus->data;
    q.front = q.front->next;
    if (q.front == nullptr) q.rear = nullptr;
    delete hapus;
    return true;
}
```



Gambar 6. Front dan rear pada linked queue.

4. Kegiatan Praktikum

Gunakan satu berkas bernama latihan_queue.cpp. Tampilkan front, rear, count, dan urutan queue setelah setiap operasi.

4.1 Membentuk queue

Gunakan struktur Queue dan fungsi pada Bab 2.

1. buat objek Queue dan panggil init;
2. periksa isEmpty, front, rear, dan count;
3. lakukan enqueue terhadap 10, 20, dan 30;
4. tampilkan queue; dan
5. pastikan urutan tampil adalah 10, 20, 30.

4.2 Enqueue, dequeue, dan wrap-around

Tambahkan fungsi pada Subbab 3.1 dan 3.2 ke dalam latihan_queue.cpp.

1. isi queue dengan 10, 20, 30, 40, dan 50;
2. lakukan dequeue dua kali;
3. tambahkan 60 dan 70;
4. amati rear kembali ke indeks awal;
5. tampilkan urutan 30, 40, 50, 60, 70; dan
6. periksa bahwa count sama dengan kapasitas.

4.3 Peek, penuh, dan kosong

Tambahkan fungsi peek dan clear pada Subbab 3.3 dan 3.5.

1. gunakan peek dan pastikan keadaan queue tidak berubah;
2. coba enqueue ketika queue penuh;
3. lakukan dequeue sampai queue kosong;
4. coba dequeue dan peek pada queue kosong; dan
5. pastikan operasi gagal tidak mengubah queue.

5. Tugas Praktikum

Buat sistem antrean layanan mahasiswa menggunakan circular queue. Seluruh program disimpan dalam modul6_antrean.cpp.

1. menambahkan mahasiswa ke antrean;
2. menampilkan seluruh antrean;
3. melihat mahasiswa berikutnya tanpa menghapusnya;
4. melayani mahasiswa paling depan;
5. menolak NIM yang sudah ada;
6. menampilkan jumlah antrean;
7. menangani antrean kosong dan penuh; dan
8. mengosongkan antrean.

5.1 Kerangka program

Salin kerangka berikut ke modul6_antrean.cpp. Definisi fungsi ditambahkan setelah main pada berkas yang sama.

```
#include <iostream>
#include <string>

const int KAPASITAS = 20;

struct Mahasiswa {
    std::string nim;
    std::string nama;
    std::string keperluan;
};

struct QueueMahasiswa {
    Mahasiswa data[KAPASITAS];
    int front;
    int rear;
    int count;
};

void init(QueueMahasiswa& q);
bool isEmpty(const QueueMahasiswa& q);
bool isFull(const QueueMahasiswa& q);
bool nimTersedia(const QueueMahasiswa& q, const std::string& nim);
bool enqueue(QueueMahasiswa& q, const Mahasiswa& data);
bool dequeue(QueueMahasiswa& q, Mahasiswa& dataKeluar);
bool peek(const QueueMahasiswa& q, Mahasiswa& dataDepan);
void tampilkan(const QueueMahasiswa& q);
void clear(QueueMahasiswa& q);
```

```
int main() {
    QueueMahasiswa antrean;
    init(antrean);
    int pilihan;

    do {
        std::cout << "1. Tambah  2. Tampilkan  3. Berikutnya\n";
        std::cout << "4. Layani  5. Jumlah     6. Kosongkan\n";
        std::cout << "7. Keluar\n";
        std::cin >> pilihan;
        // panggil fungsi sesuai pilihan
    } while (pilihan != 7);

    return 0;
}
```

5.2 Fungsi yang dilengkapi

Lengkapi operasi QueueMahasiswa dengan pola circular queue integer. Pencarian NIM memeriksa count elemen mulai dari front.

```
bool nimTersedia(const QueueMahasiswa& q,
                 const std::string& nim) {
    // lengkapi
    return false;
}

bool enqueue(QueueMahasiswa& q, const Mahasiswa& data) {
    // tolak queue penuh dan NIM duplikat
    // lengkapi
    return true;
}

bool dequeue(QueueMahasiswa& q, Mahasiswa& dataKeluar) {
    // lengkapi
    return true;
}

bool peek(const QueueMahasiswa& q, Mahasiswa& dataDepan) {
    // lengkapi
    return true;
}

void tampilkan(const QueueMahasiswa& q) {
    // lengkapi
}
```

6. Pengujian

Jalankan program dengan seluruh skenario berikut. Catat hasil aktual dan perbaiki program jika hasilnya tidak sesuai.

No.	Skenario	Hasil yang diharapkan
1	Queue baru	count == 0
2	Enqueue pertama	front == rear == 0
3	Enqueue beberapa data	Urutan FIFO benar

No.	Skenario	Hasil yang diharapkan
4	Peek	Data depan diperoleh, queue tetap
5	Dequeue	Data paling depan keluar
6	Enqueue saat penuh	Ditolak
7	Dequeue saat kosong	Ditolak
8	Wrap-around	Indeks kembali ke awal
9	Tambah NIM duplikat	Ditolak
10	Clear	count == 0

6.1 Diagnosis kode

Tentukan kesalahan, akibat, dan perbaikan setiap potongan kode berikut.

```
// A: rear tidak bergerak melingkar
++q.rear;
q.data[q.rear] = nilai;

// B: front dipindahkan sebelum data diambil
q.front = (q.front + 1) % KAPASITAS;
nilai = q.data[q.front];

// C: count tidak diperbarui
q.data[q.rear] = nilai;

// D: gagal saat front > rear
for (int i = q.front; i <= q.rear; ++i)
    std::cout << q.data[i];

// E: peek mengubah queue
nilai = q.data[q.front];
q.front = (q.front + 1) % KAPASITAS;
```